

JAVA SCRIPT

Novi Sad, 2026

JAVA SCRIPT

- Skriptni programski jezik (*da li je još uvek samo to?*)
- Inicijalno korišćen za povećanje interaktivnosti na klijentskoj strani (ugradnjom u HTML stranice)
- Spada u grupu interpreterskih jezika
- Kada se radi o kodu na klijentskoj (*front-end*) strani važno je zapamtiti da se izvršava u okruženju web čitača (browsera)

JAVA SCRIPT

- JavaScript je dinamički, višeparadigmatski jezik (*imperativno/proceduralna, objektna i funkcionalna paradigma*) koji se izvršava u pregledaču (browseru), ali sada i na serverima (Node.js).
- Više nije samo "jezik za animacije na sajtovima" — to je jedan od najraširenijih jezika na svetu.
- Dinamički tipiziran — tip se određuje u toku izvršavanja
 - kod deklaracije promenljivih se ne navodi tip promenljive
- Single-threaded sa *event loop* modelom za asinhronost
- Postoji biblioteka ugrađenih standardnih funkcija i objekata
- Kod se tipično piše kao *event handling* funkcije, koje reaguju na događaje koje proizvodi korisnik, ili iz nekog drugog izvora

JAVA SCRIPT

- Specifikacija jezika je open-source i standard se usklađuje u redovnim intervalima (pre nije bilo tako, pa je dolazilo do razmimoilaženja u verzijama i interpretaciji standarda)
- U novije vreme se intenzivno koristi i za razvoj serverske strane aplikacija (prelomni momenat je bio usvajanje ES 6 standarda)
 - Pre ES6: jedino var, callback pakao, prototipno nasleđivanje bez “sintaksnog zaslađivanja”
 - ES6+: let/const, klase, moduli, arrow funkcije, destrukuriranje, Promise, async/await
 - Savremeni projekti, biblioteke i razvojna okruženja (*frameworks*) pišu se isključivo u ES6+ stilu
- Danas prosto ne možete kreirati web sadržaj, a da barem za neke funkcionalnosti ne koristi JavaScript

JAVA SCRIPT

- Interpretiran, ali i JIT-kompajliran u modernim engine-ima:
V8 (Chrome, Edge, Brave, Opera), SpiderMonkey(Firefox), JSCoreEngine(Safari)
 - *Initial Interpretation*: Kada se kod učitava, izvršno okruženje ga izvršava linu po liniju (interpretira)
 - *Profiling & Monitoring*: Profiler izvršnog okruženja konstantno prati izvršavanje evidentirajući tipove podataka, upotrebu varijabli, koji blokovi koda se često koriste
 - *Kompajliranje*: Kada se detektuje da se neki deo koda često poziva (“hot”), “uskače” JIT kompajler, prevodeći taj deo koda u optimizovan mašinski kod, tako da se naredni pozivi izvršavaju na već kompajliranom kodu.
 - *Execution & De-optimization*: CPU direktno može da izvršava ovaj mašinski kod (značajno ubrzanje). Pošto izvršno okruženje i dalje prati izvršavanje koda, ako se u međuvremenu promeni, moguće je da se okruženje vrati na interpretirani režim, a optimizuje izvršavanje nekog drugog dela.

KAKO SE JAVASCRIPT POVEZUJE SA HTML STRANICOM?

- Skript kod se može nalaziti (a danas najčešće i jeste) u drugoj datoteci, međutim mora se pozvati iz odgovarajuće HTML datoteke

```
<head>
```

```
...
```

```
<script type="text/javascript" src="filename.js" />
```

```
</head>
```


KAKO SE JAVASCRIPT POVEZUJE SA HTML STRANICOM?

Može i navođenjem taga `<script>` (najčešće u `<head>` elementu) specificira se script kod koji se pokreće direktno u browseru

```
<script type="text/javascript">  
    //JAVA SCRIPT CODE  
</script>
```

- Sve između tagova `<script>` i `</script>` *browser* smatra kodom skripta
- `<script>` tag se može javiti bilo gde u HTML dokumentu (danas više nije uobičajeno)
 - Script u `<body>` sekciji se izvršava prilikom iscrtavanja stranice
 - Script u `<head>` sekciji najčešće sadrži kod koji se poziva iz elemenata u stranici

PROMENLJIVE (VARIABLE) U JS

- Kao i u drugim jezicima varijabla služi za čuvanje podataka tokom izvršavanja programa (skripta)
- dovoljno je napisati npr. `a=5`
- ipak se preporučuje upotreba eksplicitne deklaracije pomoću ključnih reči `let`, `const` ili `var`.

Upotreba `var` se više ne preporučuje u ES 6 + kodu (problematičan i neintuitivan opseg važenja)

`var x = 2 ; // function-scope` (ili *global scope* ako je izvan funkcije)

`let x1 = 2 ; // block-scope` varijabla - vrednost je naknadno moguće menjati

`const x2 = 2 ; // block-scope` konstantna - vrednost se ne može menjati

TIPOVI PODATAKA

- Iako ih mi ne navodimo tipovi podataka, kao i za svaki drugi programski jezik moraju postojati
 - **Number** - celobrojni ili realan broj
 - **String** - objekat koji reprezentuje niz karaktera (može se pisati kao “tekst” ili ‘tekst’)
 - **Boolean** - true/false
 - **Null** - specijalna vrednost koja govori da je varijabla prazna
 - **Undefined** - specijalna vrednost koju ima varijabla koja je deklarisan, ali joj nikada nije dodeljena vrednost
 - **Object** - reprezentuje složeni tip podataka - objekte sa svojim propertyjima u obliku *key: value*. Mogu biti ugrađeni (Array, Math, Date, Function, String, Number), ili korisnički definisani.
 - **Array** - kolekcija podataka u formi niza
 - **Function** - u JS su funkcije objekti koji se mogu pozvati da izvrše određeni blok koda

KORISNIČKI DEFINISANI OBJEKTI

- Mogu se kreirati na nekoliko načina

- pomoću *object literal*

```
const osoba = {  
  ime: "Neko",  
  prezime: "Nepoznat",  
};
```

- korišćenjem *new Object* sintakse

```
const osoba2 = new Object({  
  ime: "Neko",  
  prezime: "Nepoznat",  
});
```

Može iako je const, jer se menja property objekta, a ne referenca na sam objekat

```
const osoba3 = new Object();  
osoba3.ime = "Neko";  
osoba3.prezime = "Nepoznat";
```


UPOTREBA CONST NA OBJEKTE

```
const user = { name: "Alice" };
```

```
user.name = "Bob"; //  Može: menja se property postojećeg objekta
```

```
user.age = 25; //  Može: postojećem objektu se dodaje property
```

```
//  Ne može! Pokušaj dodele novog objekta postojećoj const referenci
```

```
user = { name: "Charlie" };
```

U JAVASCRIPTU MNOGI TIPOVI SU OBJEKTI

- Varijable određenih tipova su objekti:
 - *Object* su objekti
 - *Math* su objekti
 - *Function* su objekti
 - *Date* su objekti
 - *Array* su objekti
 - *Map* su objekti
 - *Set* su objekti
 - ...

JS NIZOVI

- Niz u JavaScript-u je dinamički, uređeni skup elemenata indeksiranih celim brojevima.
- Za razliku od C ili Jave, JS nizovi su heterogeni (mogu mešati tipove), automatski menjaju veličinu
- Često korišćena kolekcija
- Slično kao u Javi, sadrži elemente, ali ima i attribute i metode (implementirani kao objekti sa posebnim ponašanjem)
 - atribut *length* sadrži trenutnu veličinu niza
 - metode omogućavaju dodavanje, uklanjanje, i manipulaciju elementima

JS NIZOVI

- Ključne osobine:
 - Uređen — redosled je garantovan, indeks počinje od 0
 - Dinamički — length se menja automatski, nema fiksne veličine
 - Heterogen — elementi mogu biti različitih tipova u istom nizu
 - Dozvoljava duplikate — ista vrednost može biti na više pozicija
 - Brzina pristupa:
 - Pristup po indeksu: $O(1)$ — niz[i] je trenutni lookup
 - Pretraga po vrednosti: $O(n)$ — includes(), indexOf(), find() prolaze linearno kroz niz

JS NIZOVI - KREIRANJE

// Kreiranje niza Array objektom

```
const niz1 = new Array();
```

```
niz1[0] = 5;
```

```
let niz2 = new Array(1, 2, 3, 4);
```

// Kreiranje niza "JSON" sintaksom

```
const niz3 = []; //kreira prazan niz
```

```
niz3.push(4); // niz3 je sada [4]
```

//direktni upis preko indeksa - iako može, potencijalno problematično

```
niz3[3] = 6; // niz3 je sada? -> [4, undefined, undefined, 6] - pažnja!
```

// Array.from() — iz iterabilnog ili array-like objekta

```
const izStringa = Array.from("hello"); // ["h","e","l","l","o"]
```

```
const izSeta = Array.from(new Set([1,2,3])); // [1, 2, 3]
```

```
const generisan = Array.from({ length: 5 }, (_, i) => i * 2);
```

```
// [0, 2, 4, 6, 8]
```

JS NIZOVI - PRISTUP I PRIMER OPERACIJA

```
const ocene = [7, 9, 6, 10, 8, 9, 5];

// Pristup – O(1)
ocene[0]; // 7 – prvi element
ocene[ocene.length-1]; // 5 – poslednji (ili ocene.at(-1) u ES2022)
ocene.at(-2); // 9 – drugi od kraja

// Pretraga – O(n), prolazi kroz ceo niz
ocene.includes(10); // true
ocene.indexOf(9); // 1 – prva pojava
ocene.lastIndexOf(9); // 5 – poslednja pojava
ocene.findIndex(o => o < 7); // 2 – indeks prve ocene ispod 7

// Veličina i proveravanje
ocene.length; // 7
Array.isArray(ocene); // true – jedini pouzdan način proveravanja
```


JAVASCRIPT NIZOVI - METODE

- Array sadrži veliki broj metoda koje olakšavaju pristup i manipulaciju podacima
 - push() - dodaje novi niz na kraj niza
 - pop() - uklanja poslednji element
 - concat() -spaja elemente nizova u jedan niz
 - shift() - uklanja prvi element iz niza
 - unShift() - dodaje novi element umetanjem na početak niza
 - reverse() - obrće redosled elemenata u nizu
 - slice() - kopira deo niza u novi niz
 - splice() - dodaje element na određeno mesto
 - toString() - pretvara elemente niza u string
 - valueOf() - vraća primitivnu vrednost datog objekta
 - indexOf() - vraća prvi indeks traženog elementa u nizu (ako postoji), ili -1 ako ne postoji
 - lastIndexOf() - vraća najveći indeks traženog elementa u nizu, ili -1 ako ne postoji
 - join() - kombinuje elemente niza u jedan string
 - sort() - sortira elemente niza po zadatom kriterijumu

JS - OPERATORI

- Aritmetički: `+` `-` `*` `/` `%` `++` `--`

```
x = 5;
```

```
y = x * 4;
```

```
z = y % 5;
```

- Dodele: `=` `+=` `-=` `*=` `/=` `%=`

```
y += 5;            y=y+5;
```

- Operator `+` ima posebno značenje kada su operandi stringovi:

```
a = "Pera";
```

```
b = "Car";
```

```
c = a + b;
```

- Kada sabiramo stringove i brojeve, rezultat je string

JS - RELACIONI OPERATORI

- Relacioni: `==` `===` `!=` `<` `<=` `>` `>=`

```
let x = 5;
```

```
if (x == 5)
```

```
    console.log("x je jednako 5");
```

- Operator `===` će porediti i vrednost i tip:

```
let x = 5;
```

```
if (x === "5") // == bi vratilo true
```

```
    console.log("x je string sa sadržajem 5");
```

- Rezultat relacionih operatora je logička vrednost tačno (true) ili netačno (false)

JS - LOGIČKI OPERATORI

- Logički: **&& | | !**
- Rezultat logičkih operatora je tačno (true) ili netačno (false)
- Operandi logičkih operatora su logički izrazi

&&	false	true
false	false	false
true	false	true

	false	true
false	false	true
true	true	true

!	
false	true
true	false

INICIJALIZACIJA NA PODRAZUMEVANU VREDNOST

- Operator `||` koristi se i za inicijalizaciju na podrazumevanu vrednost kada je vrednost sa leve strane *"falsy"* (`false`, `0`, `""`, `null`, `undefined`, `NaN`)

```
let userName = providedName || "Neprijavljeni korisnik";
```

- Ipak kada su `0`, `""` i `false` moguće i potrebne kao vrednosti promenljive bolje je koristiti *nullish coalescing operator ??*
 - ovaj operator samo vrednosti `null` i `undefined` smatra za nevalidne i zamenjuje ih datom podrazumevanom vrednošću

```
let age = undefined;  
let userAge = age ?? 25;
```

KONTROLA TOKA

- if ... else
- switch
- for, for-in, for-of
- while
- do while
- break
- continue

IF - ELSE

- Opšta sintaksa:

```
if (uslov_1) {  
    telo_1  
} else if (uslov_2) {  
    telo_2  
} else {  
    telo_3  
}
```

IF - ELSE PRIMER

```
if (poeni >= 91)
    ocena = 10;
else if (poeni >= 81)
    ocena = 9;
else if (poeni >= 71)
    ocena = 8;
else if (poeni >= 61)
    ocena = 7;
else if (poeni >= 51)
    ocena = 6;
else ocena = 5;
```


TERNARNI USLOVNI OPERATOR

- Sintaksa

```
promenljiva = (uslov) ? vrednost1 : vrednost2
```

- Kao i u Javi zamena za:

```
if (uslov)
```

```
    promenljiva = vrednost1;
```

```
else
```

```
    promenljiva = vrednost2;
```

SWITCH

- Izraz u switch() izrazu mora da proizvede celobrojnu vrednost.
- Ako ne proizvodi celobrojnu vrednost, ne može da se koristi switch(), već if()!
- Ako se izostavi break; propašće u sledeći case.
- Kod default izraza ne mora break to se podrazumeva.

SWITCH PRIMER

```
switch (a)
{
    case 1:
    case 2: i = j + 6;
        break;
    case 3: i = j + 14;
        break;
    default: i = j + 8;
}
```

PETLJE - FOR

- **for** - za “brojačke” petlje kod kojih se može unapred zadati broj ponavljanja, odnosno uslov završetka

```
for (initialization; testing; increment/decrement)
{
    statement(s) ;
}
```

primer:

```
for (let x = 2; x <= 10; x++) {
    console.log("Vrednost x:" + x);
}
```


PETLJE - FOR .. OF

- **for-of** - verzija petlje koja omogućava jednostavno iteriranje kroz elemente koji se nalaze u *iterable objects* (Array, String, Map, Set...)

```
for (variableName of iterableObject)
{
    statement(s);
}
```

// primeri:

```
const a = [ 1, 2, 3, 4, 5 ];
```

```
for (const item of a) {
    console.log(item);
}
```

```
const str = "Hello";
```

```
for (const char of str) {
    console.log(char);
}
```

PETLJE - FOR .. IN

- **for-in** - verzija petlje koja omogućava jednostavno iteriranje **kroz objekte**

```
for (variableName in Object) {...}
```

```
// primer
```

```
const car = {  
  make: "Toyota",  
  model: "Corolla",  
  year: 2020  
};
```

```
for (let key in car) {  
  console.log(`${key}: ${car[key]}`);  
}
```

Jednostavno, ali iterira i kroz property-je *prototype* objekta, pa je često bolje koristiti alternativu: `Object.keys()`; / `Object.entries()`;

Ne koristiti `for ... in` za iteriranje kroz nizove!

PETLJE - WHILE

- **while** - petlje kod kojih se na početku svakog novog ciklusa izračunava vrednost logičkog uslova, koji mora biti true da bi se sledeći ciklus izvršio.

```
while (boolean condition)
```

```
{  
    loop statements...  
}
```

- **do-while** - Kao i while, ali je provera uslova na kraju svakog ciklusa (sigurno se obavlja barem jedan prolazak kroz telo petlje)

```
do
```

```
{  
    statements..  
}
```

```
while (condition);
```

TEMPLATE LITERALI

- Stringovi koji podržavaju umetanje izraza i višelinijski tekst, bez zamornog spajanja stringova
 - koriste se **backtick** navodnici `.

```
const user = "Ana";  
const score = 42;
```

```
// Stari stil:  
console.log("Zdravo, " + user + "! Poeni: " + score);
```

```
// Template literal:  
console.log(`Zdravo, ${user}! Poeni: ${score}`);
```

```
// Izraz unutar ${}:  
console.log(`Rezultat: ${score > 50 ? "položio" : "nije položio"}`);
```

```
// Višelinijski string:  
const poruka = `Poštovani ${user},  
Vaš rezultat je ${score},  
Hvala što ste učestvovali.`;
```


DESTRUKTURISANJE

- Destrukturisanje omogućava izvlačenje vrednosti iz **objekata i nizova** u lokalne promenljive jednim izrazom. Poznat vam je iz Pythona (tuple unpacking)
 - u JS-u je raznovrsniji (ne radi samo po poziciji, dozvoljava podrazumevane vrednosti, dozvoljava da se broj elemenata koji se destrukturišu ne poklapa sa brojem varijabli...)
- Omogućava i lepu „prečicu“ kada treba „raspakovati“ *property*je objekta u varijable sa istim nazivom (zgodno na primer kada treba „raspakovati“ request objekat).
- Dovoljno je u izrazu dodele navesti varijable koje imaju isti naziv kao traženi property-ji

```
const myObject = { prop1: "lorem", prop2: "ipsum" };  
let { prop1, prop2 } = myObject;  
console.log(prop1); // "lorem"  
console.log(prop2); // "ipsum"
```

DESTRUKTURISANJE

```
const response = {  
  status: 200,  
  data: { name: "Nikola", age: 22 },  
  ok: true  
};
```

// Destrukturiranje objekta u varijable

```
const { status, data: { name, age }, ok } = response;  
console.log(status, name, age); // 200 "Nikola" 22
```

// Sa preimenovanjem i podrazumevanim vrednostima

// `status` property objekta se smešta u varijablu `httpStatus`

```
const { status: httpStatus = 500, message = "Nema poruke" } =  
response;
```

// Destrukturisanje niza

```
const [first, second, ...rest] = [10, 20, 30, 40, 50];  
console.log(first); // 10  
console.log(rest); // [30, 40, 50]
```


DESTRUKTURISANJE

- Često je moguće da je ulazni parametar funkcije bude neki objekat (potencijalno sa mnogo propertyja), što je i zgodnije nego da korisnici pamte redosled parametara.
- Ako nama trebaju samo određeni property-ji, prosto destrukturiramo ulazni parametar izvlačeći samo one koji nam stvarno trebaju u funkciji

```
// U parametrima funkcije - funkciji se kao argument prosleđuje objekat  
// radi jednostavnosti, iz objekta "izvučemo" one koji nam trebaju
```

```
function display({ name, age }) {  
    console.log(`${name} ima ${age} godina`);  
}
```

```
function removeBreakpoint({ url, line, column }) {  
    // ...  
}
```

SET I MAP — KADA I ZAŠTO UMESTO NIZA?

- Koliko košta provera da li vrednost postoji?

```
const niz = [1, 2, 3, ..., 1000000];  
niz.includes(999999); // O(n) – prolazi od početka do kraja
```

```
const skup = new Set([1, 2, 3, ..., 1000000]);  
skup.has(999999); // O(1) – direktan hash lookup
```

```
const mapa = new Map();  
mapa.set("kljuc", vrednost);  
mapa.has("kljuc"); // O(1) – bez obzira na veličinu mape
```


SET - KOLEKCIJA JEDINSTVENIH VREDNOSTI

- Set garantuje jedinstvenost:
isti element ne može biti dodat dva puta.
- Jednakost se proverava kao strogu referencijalna
jednakost (===) za objekte, a po vrednosti za primitive.
- Skupovne operacije — presek, unija, razlika — bile su
moguće i ranije, ali ES2025 uvodi native metode:
intersection(), union(), difference().

SET - PRIMER

```
// Osnovno korišćenje
const dozvoljeniStatusi = new Set(["aktivan", "neaktivan",
"suspendovan"]);

dozvoljeniStatusi.add("obrisan");
dozvoljeniStatusi.add("aktivan");    // duplikat – ignorisan
dozvoljeniStatusi.size;              // 4, ne 5

dozvoljeniStatusi.has("aktivan");    // true – O(1)
dozvoljeniStatusi.has("izbrisan");   // false – O(1)

dozvoljeniStatusi.delete("suspendovan");

// Iteracija – čuva redosled umetanja
for (const status of dozvoljeniStatusi) {
    console.log(status);
}
```


SET - PRIMER

```
// Uklanjanje duplikata iz niza
```

```
const tagovi = ["js", "web", "js", "css", "web", "js"];
```

```
const jedinstveniTagovi = [...new Set(tagovi)];
```

```
// ["js", "web", "css"]
```

```
// Skupovne operacije za verziju pre JS2025 - lako se napišu
```

```
const a = new Set([1, 2, 3, 4]);
```

```
const b = new Set([3, 4, 5, 6]);
```

```
const presek      = new Set([...a].filter(x => b.has(x))); // {3, 4}
```

```
const unija       = new Set([...a, ...b]);                //
```

```
{1,2,3,4,5,6}
```

```
const razlika     = new Set([...a].filter(x => !b.has(x))); // {1, 2}
```

MAP — MAPA SA KLJUČEVIMA BILO KOG TIPa

- Map je uređena kolekcija parova ključ-vrednost.
- Ključ može biti bilo koji tip: string, broj, objekat, čak i funkcija.
- Čuva redosled umetanja i ima $O(1)$ operacije.
- Map vs Object — kada koristiti šta?
 - Map: dinamički ključevi, nestring ključevi, česta add/delete, size potreban
 - Object: statička struktura, JSON serijalizacija, poznat skup polja
 - **VAŽNO:** Objekat se ne može direktno serijalizovati u JSON format, ako sadrži Map — koristite [...map.entries()] ili Object.fromEntries(map) pre JSON.stringify().

MAP - PRIMER

```
// Ključevi mogu biti bilo šta
const cache = new Map();
// Ključ je objekat koji opisuje npr. poziv koji smo obavili
const requestKey = { method: "GET", url: "/api/users" };
cache.set(requestKey, { data: [], cachedAt: Date.now() });
cache.get(requestKey); // vraća vrednost po referenci objekta
```

```
// Čitljiviji oblik inicijalizacije
const httpStatusi = new Map([
  [200, "OK"],
  [201, "Created"],
  [400, "Bad Request"],
  [401, "Unauthorized"],
  [404, "Not Found"],
  [500, "Internal Server Error"],
]);
```

```
httpStatusi.get(404); // "Not Found" — O(1)
httpStatusi.has(418); // false
```

PROMISE OBJEKTI

- Promise je objekat koji predstavlja buduću vrednost — rezultat operacije koja još nije završena.
- Tri stanja: *pending*, *fulfilled*, *rejected*.
- Koriste se da “sačekaju” izvršavanje dugotrajnih operacija
 - ako je izvršavanje uspešno onda promise objekat prelazi iz *pending* u *fulfilled* stanje
 - ako je neuspešno - onda se prelazi u *rejected* stanje (omogućava obradu greške)
- Fetch API za komunikaciju sa HTTP serverima oslanja se na Promise objekte kako bi enkapsulirao rezultat komunikacije sa serverom

PROMISE OBJEKTI

- Za Promise je tipično potrebno napisati *event handling* funkcije
- `resolve` i `reject` su statičke metode
- Primer promise objekta (simulira rezultat)

```
const deliveryPromise = new Promise((resolve, reject) => {  
  let itemDelivered = true; // Simulate the outcome  
  
  if (itemDelivered) {  
    resolve("Your package has arrived!"); // Success path  
  } else {  
    reject("Delivery failed due to bad weather."); // Failure path  
  }  
});
```

PROMISE OBJEKTI

- Konzumiranje Promise-a — `.then()/ .catch()/ .finally()`.
- Primer konzumiranja prethodnog promise objekta
- *then* očekuje callback funkciju koja će isprocesirati podatke koje Promise stavlja na raspolaganje kada pređe u stanje *fulfilled*
- *catch* callback funkciju koja će isprocesirati podatke/ grešku koju Promise stavlja na raspolaganje kada pređe u stanje *rejected*

```
deliveryPromise.then( (message) => {  
    console.log("Success: " + message); // Runs if resolve() was called  
}  
) .catch( (error) => {  
    console.log("Error: " + error); // Runs if reject() was called  
}  
);
```


PROMISE OBJEKTI

- Primer upotrebe
- `fetch()` vraća Promise objekat - kada se uspešno rezolvira, *response* enkapsulira odgovor servera, a da bi se pročitao sadržaj *body*-ja, pozove se neka od metoda - ovde `.json()` - koja i sama vraća drugi promise (parsiranje može da potraje), ako se uspešno rezolvira vraća *data*

```
fetch("https://github.com")
  .then((response) => response.json())//Converts raw data to JSON (returns another promise)
  .then((data) => {
    console.log("User data received: ", data);// Logs the final data
  })
  .catch((error) => {
    console.error("Could not fetch data: ", error);// Catches network errors
  });
```